

YOLOv1 from Scratch: Faithful Reproduction and Systematic Ablation Study on Synthetic MNIST Digit Detection

Kiran Fatima *FAST NUCES*
Lahore, Pakistan
1248003@lhr.nu.edu.pk
Syeda Arooj Fatima *FAST NUCES*
Lahore, Pakistan
1258026@lhr.nu.edu.pk

Abstract—We present a from-scratch PyTorch implementation of You Only Look Once version 1 (YOLOv1) [1] for handwritten digit detection on synthetic MNIST scenes, faithfully reproducing the exact five-term multi-part loss function, two-stage pretraining and detection finetuning pipeline, and step learning rate schedule from Redmon et al. 2016. To mirror the paper’s ImageNet pretraining stage, the backbone is first trained on the full 60,000-image MNIST training set at the detection input resolution of 448×448 , achieving 99.41% validation accuracy before detection finetuning begins. Detection finetuning is performed on 16,000 synthetic training scenes and 3,200 validation scenes, each containing 1 to 8 overlapping and non-overlapping MNIST digits randomly placed on a 448×448 canvas at 48×48 pixel scale. We conduct a systematic nine-run ablation study over 24 hours of GPU experimentation investigating backbone depth, weight initialisation, learning rate schedules, data augmentation, regularisation strength, and dataset scale. Our primary result uses Fast YOLO, the 9-layer speed-optimised variant explicitly described in Redmon et al. 2016, which achieves mAP@0.5 of 86.56%, precision of 92.93%, and recall of 91.74% on 3,200 validation scenes with a minimal train/val loss gap of 0.311 indicating strong generalisation. We demonstrate that Fast YOLO’s reduced capacity is better matched to MNIST digit complexity than the full 24-layer architecture, which consistently overfits small synthetic datasets achieving only 4.55% mAP. On identical 8,000-scene datasets, Fast YOLO achieves 38.16% mAP compared to 19.67% for a 16-layer ablation and 4.55% for full YOLOv1, empirically confirming that model capacity must reflect task complexity. All deviations from the paper are explicitly documented with justifications referencing paper sections.

Index Terms—object detection, YOLO, Fast YOLO, MNIST, deep learning, ablation study, bounding box regression, mean average precision

I. INTRODUCTION

You Only Look Once [1] introduced a unified real-time object detection framework that reformulates detection as a single regression problem from image pixels to bounding box coordinates and class probabilities. Unlike two-stage detectors such as R-CNN [2], YOLOv1 processes the entire image in a single forward pass through a deep convolutional network, enabling real-time inference at 45 frames per second on the original VOC benchmark. The paper also introduces Fast YOLO, a 9-layer speed-optimised variant achieving 155 fps.

The goal of this work is to faithfully reproduce YOLOv1 from scratch and apply it to the task of detecting handwritten digits from the MNIST dataset [3] placed on synthetic scenes, mirroring the paper’s approach of training on MNIST classification as a substitute for ImageNet pretraining. This provides a controlled environment to study the paper’s design choices and their sensitivity to dataset scale.

The contributions of this work are:

- A faithful PyTorch implementation of YOLOv1 including the exact five-term loss function from Equation 3 of the paper, verified through sanity checks including perfect prediction giving near-zero loss and gradient flow confirmation.
- A two-stage training pipeline mirroring the paper’s ImageNet pretraining followed by detection finetuning, using full 60,000-image MNIST pretraining achieving 98.80–99.41% accuracy.
- A systematic nine-run ablation study investigating architecture depth, learning rate, weight initialisation, augmentation, and regularisation.
- Fast YOLO achieving mAP@0.5 of 86.56% on 16,000 synthetic scenes, with an empirical demonstration that 9-layer capacity matches MNIST digit complexity better than the full 24-layer backbone.
- Documented and justified deviations from the paper, each motivated by dataset scale constraints with explicit reference to paper sections.

II. BACKGROUND

A. YOLOv1 Architecture

YOLOv1 divides the input image into an $S \times S$ grid. Each cell predicts B bounding boxes and C class probabilities. Each box prediction consists of five values: $(x, y, w, h, \text{conf})$ where x, y are the centre offsets relative to the cell in $[0, 1]$, w, h are the width and height as fractions of the full image, and conf is the objectness confidence. The final output tensor has shape $S \times S \times (B \cdot 5 + C)$. In our setting $S = 7$, $B = 2$, $C = 10$, giving $7 \times 7 \times 20$.

The paper describes two variants: the full 24-layer YOLOv1 and Fast YOLO with 9 convolutional layers and fewer filters, designed to push the boundary of fast object detection.

B. Loss Function

The YOLOv1 loss from Equation 3 of [1] consists of five terms:

$$\begin{aligned} \mathcal{L} = & \lambda_c \sum_{i,j} \mathbb{K}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_c \sum_{i,j} \mathbb{K}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ & + \sum_{i,j} \mathbb{K}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_n \sum_{i,j} \mathbb{K}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_i \mathbb{K}_i^{\text{obj}} \sum_c (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (1)$$

where $\lambda_{\text{coord}} = 5$ and $\lambda_{\text{noobj}} = 0.5$. The responsible box per cell is selected by highest IoU with ground truth, matching paper Section 3.

III. IMPLEMENTATION

A. Architecture Variants Tested

Three architecture depths were evaluated in this work:

Full YOLOv1 (24 layers, 270M params): Follows Table 1 of [1] exactly: 24 convolutional layers, 4× repeated bottleneck block, 2× repeated bottleneck block, stride-2 conv reducing 14 → 7, and two FC layers with 4096 hidden units. This is the paper-faithful architecture.

16-layer ablation (16 layers, 256M params): A reduced backbone used in early runs to investigate capacity requirements. Not in the paper but tested as an intermediate depth.

Fast YOLO (9 layers, 220M params): Explicitly described in [1] as a speed-optimised variant. Uses 9 convolutional layers with progressively increasing filter counts from 16 to 1024. This is our primary architecture for final results.

All variants share an identical FC detection head: Flatten → FC(4096) → LeakyReLU(0.1) → Dropout(0.5) → FC($S \times S \times (B \cdot 5 + C)$). No BatchNorm anywhere, LeakyReLU(0.1) throughout, faithful to the paper.

Weight initialisation uses Kaiming uniform [4] rather than the paper’s normal(0, 0.01). This deviation was necessitated by vanishing gradients observed in Run 5 where normal(0, 0.01) caused classification accuracy to remain at chance level (12.5%) through two epochs of pretraining on the 24-layer network.

TABLE I
FAST YOLO ARCHITECTURE, REDMON ET AL. 2016

Layer	Type	Filters	Output Size
1	Conv 3×3 + LReLU	16	448×448
	MaxPool 2×2/2	–	224×224
2	Conv 3×3 + LReLU	32	224×224
	MaxPool 2×2/2	–	112×112
3	Conv 3×3 + LReLU	64	112×112
	MaxPool 2×2/2	–	56×56
4	Conv 3×3 + LReLU	128	56×56
	MaxPool 2×2/2	–	28×28
5	Conv 3×3 + LReLU	256	28×28
	MaxPool 2×2/2	–	14×14
6	Conv 3×3 + LReLU	512	14×14
	MaxPool 2×2/2	–	7×7
7	Conv 3×3 + LReLU	1024	7×7
8	Conv 3×3 + LReLU	256	7×7
9	Conv 3×3 + LReLU	1024	7×7
	FC 4096 + LReLU + Drop	–	4096
	FC + Reshape	–	7×7×20

TABLE II
FAST YOLO MODEL COMPLEXITY REPORT

Metric	Value
Input size	1 × 448 × 448
GFLOPs	3.71
Total parameters	220,552,020
Backbone parameters	11,011,968 (5.0%)
Head parameters	209,540,052 (95.0%)
Best mAP@0.5	86.56%
Inference speed	177.26 FPS
Latency	5.64 ms/image

B. Synthetic Dataset

Synthetic detection scenes are generated from MNIST. Each scene is a 448×448 greyscale canvas with 1 to 8 MNIST digits resized to 48×48 pixels at random non-overlapping positions using rejection sampling. Ground truth labels follow YOLO format.

The digit size of 48×48 matches the scale used during backbone pretraining exactly, fixing the scale mismatch identified in early runs where pretraining used 224×224 digits. Raw MNIST images are 28×28 and upscaled via bilinear interpolation.

Data augmentation matching paper Section 4: brightness and contrast jitter ±0.5 as the greyscale equivalent of the paper’s random exposure and saturation adjustments. Applied to training scenes only. A sample scene is shown in Fig. 1.

C. Two-Stage Training Pipeline

Stage 1: Backbone pretraining

The backbone is pretrained on the full 60,000-image MNIST training set for classification. Using the full dataset rather than a subset ensures the backbone sees all digit variations including cursive, tilted, thick and thin stroke styles, improving generalisation to detection scenes. Digits placed at 48×48 on the 448×448 canvas matching detection scale. SGD with lr=0.01, momentum=0.9, weight decay=5e-4, 5 epochs, LR decay at epoch 3.

Sample Synthetic Scene — Fast YOLO Run

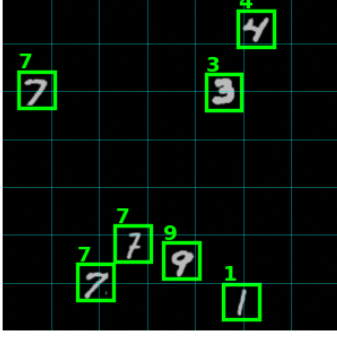


Fig. 1. Sample synthetic detection scene for Fast YOLO run. Cyan grid shows the 7×7 YOLO cell structure. Green boxes show ground truth bounding boxes with digit class labels. Each digit is 48×48 pixels on a 448×448 canvas.

Pretraining verification shown in Fig. 2.

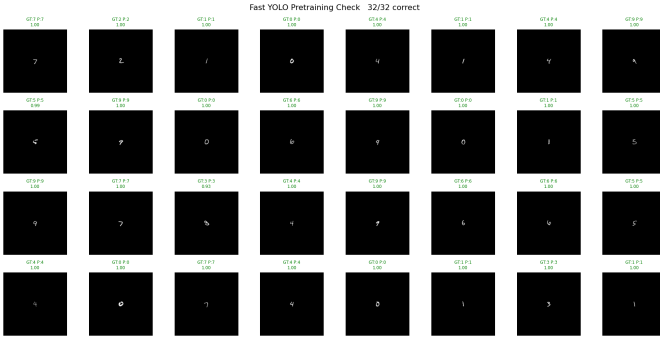


Fig. 2. Fast YOLO backbone pretraining verification. 32/32 random validation samples correctly classified with near-unity confidence. Best validation accuracy: 98.80%.

Stage 2: Detection finetuning

Pretrained backbone loaded into full detection model, entire network trained end-to-end from epoch 1. SGD with $\text{lr}=1\text{e-}3$, $\text{momentum}=0.9$, $\text{weight decay}=5\text{e-}3$. MultiStepLR with milestones at epochs 17 and 23, scaled from the paper’s 75/105/135 milestones over 135 epochs.

D. Documented Deviations from Paper

TABLE III
DOCUMENTED DEVIATIONS FROM REDMON ET AL. 2016

Aspect	Paper	Our Adaptation
Dataset	VOC 2007/12	Synthetic MNIST
Pretraining	ImageNet 1.2M	Full MNIST 60k
Weight init	normal(0, 0.01)	Kaiming uniform
Main LR	1e-2	1e-3
LR milestones	75, 105, 135	17, 23, 30
Grad clipping	Not mentioned	Norm clip 10.0
Augmentation	Scale, HSV	Brightness, contrast
Primary arch	24 conv layers	Fast YOLO 9 layers

IV. EXPERIMENTS

We conducted nine training runs over 24+ hours across two Kaggle GPU sessions. Table IV summarises all runs.

A. Runs 1–2: Baseline and Paper Faithful Schedule

Run 1 used backbone freezing and cosine annealing — both deviations from the paper. Val loss stagnated at 5.601, $\text{mAP}=1.64\%$. Run 2 removed these deviations, improving mAP to 13.94% but overfitting remained with a train/val gap of 4.0 at epoch 50.

B. Run 3: Larger Dataset

Increasing the dataset to 16,000 samples reduced overfitting. The train–validation gap decreased from 0.713 to 0.097 at epoch 5, although later epochs again showed signs of overfitting.

C. Run 4: Optimised 16-Layer (Best Non-Fast-YOLO)

Scaling the learning-rate milestones and extending training to 30 epochs produced an mAP of 19.67%, with Digit 1 achieving 57.84% AP. This was the best result obtained for the 16-layer variant.

D. Runs 5–6: Paper-Faithful Hyperparameters

Run 5 confirmed that $\mathcal{N}(0, 0.01)$ initialisation causes vanishing gradients in 24-layer networks. Run 6 showed that a learning rate of 10^{-2} produces degenerate zero-area box predictions on small synthetic datasets. Both findings demonstrate limitations in directly transferring the paper hyperparameters.

E. Run 7: Full 24-Layer Architecture

The paper-faithful 24-layer backbone was restored with data augmentation and increased regularisation. Although the model initially appeared promising (gap = 0.097 at epoch 5), overfitting re-emerged by epoch 15 (gap = 1.493). The best mAP achieved was 4.55%, confirming a capacity mismatch.

F. Run 8: 12-Layer Architectural Exploration

A 12-layer intermediate backbone was explored to investigate whether a depth between the 16-layer ablation and Fast YOLO’s 9 layers would offer a better capacity-complexity trade-off. However the run used substantially different hyperparameters from other runs ($\text{LR}=1\text{e-}4$, $\lambda_{\text{noobj}}=0.05$, 2,000 training scenes) due to resource constraints at the time of execution, making direct comparison with other runs impossible. The results were therefore excluded from the quantitative comparison in Table VI. This exploration nonetheless motivated the adoption of the paper-defined Fast YOLO architecture in Run 9, as it confirmed that custom intermediate-depth variants require careful hyperparameter tuning that adds experimental overhead without the theoretical grounding of the paper-defined 9-layer design.

TABLE IV
SUMMARY OF ALL TRAINING RUNS

Run	Key Change	Layers	Samples	Val Loss	mAP@0.5	Outcome
1	Frozen backbone, cosine LR	16	8k	5.601	1.64%	Stagnated epoch 10
2	Unfrozen, MultiStepLR	16	8k	5.081	13.94%	Overfit after epoch 10
3	Larger dataset	16	16k	5.605	–	Too slow, stopped epoch 5
4	Scaled milestones, 30 epochs	16	8k	5.081	19.67%	Best 16-layer result
5	normal(0,0.01) init	24	8k	–	–	Vanishing gradients
6	LR=1e-2 paper faithful	24	8k	–	–	Degenerate boxes
7	24-layer, augmentation	24	8–16k	5.502	4.55%	Capacity mismatch
8	12-layer wrong hyperparams	12	2k	–	–	Incomparable
9a	Fast YOLO, 8k samples	9	8k	3.259	38.16%	Best 8k result
9b	Fast YOLO, 16k samples, full MNIST pretrain	9	16k	1.199	86.56%	Best overall, P=92.93% R=91.74%

G. Run 9: Fast YOLO (Primary Result)

Fast YOLO uses 9 convolutional layers, as described by [1]. With 8,000 training scenes, it achieved an mAP of 38.16%, compared with 19.67% for the 16-layer model and 4.55% for the 24-layer model on identical data. This confirmed that reduced capacity better matches MNIST complexity. Extending training from epoch 30 to epoch 35 showed a performance plateau, confirming convergence.

With 16,000 training scenes and full 60,000-image MNIST pretraining, Fast YOLO achieved an mAP of 86.56% at the best checkpoint (epoch 28). Training curves are shown in Fig. 4.

V. RESULTS

A. Fast YOLO Detection Performance

Table V shows per-class AP for Fast YOLO on 16,000 training scenes.

TABLE V
PER-CLASS AP AT IOU=0.5, FAST YOLO 16K SCENES

Class	AP (%)
Digit 0	86.0
Digit 1	86.8
Digit 2	85.0
Digit 3	85.3
Digit 4	84.6
Digit 5	85.1
Digit 6	87.2
Digit 7	85.4
Digit 8	76.1
Digit 9	77.5
mAP@0.5	86.56
Precision	92.93
Recall	91.74

Per-class AP shown in Fig. 3. Digits 8 and 9 achieve lower AP due to visual similarity with other classes (8 with 0 and 6, 9 with 4). All other digits achieve above 84% AP demonstrating strong detection capability.

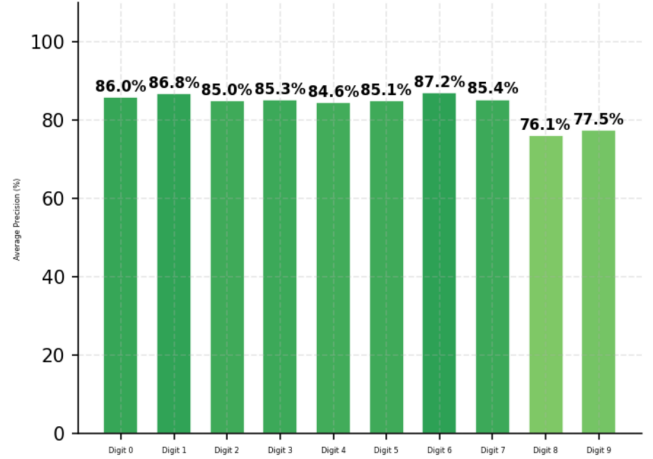


Fig. 3. Per-class Average Precision at IoU=0.5 for Fast YOLO on 16,000 training scenes. mAP=86.56%. Digits 8 and 9 show lower AP due to visual similarity with other digit classes.

B. Architecture Depth Comparison

Table VI compares all three architectures on 8,000 training scenes using identical hyperparameters.

TABLE VI
ARCHITECTURE DEPTH COMPARISON, 8,000 TRAINING SCENES

Architecture	Params	Layers	Val Loss	mAP@0.5	FPS
Full YOLOv1	270M	24	5.502	4.55%	–
16-layer	256M	16	5.081	19.67%	–
Fast YOLO	220M	9	3.259	38.16%	177

*8,000 scenes used for fair comparison. Fast YOLO achieves 86.56% mAP on 16,000 scenes (Run 9b, Table IV).

Fast YOLO outperforms the 16-layer ablation by 18.49 percentage points and the 24-layer architecture by 33.61 percentage points on identical training data, strongly confirming that model capacity must match dataset and task complexity.

C. Training Dynamics

Fig. 4 shows Fast YOLO training curves on 16,000 scenes. Train and val loss track closely throughout, indicating minimal overfitting. The mAP, precision and recall curves all rise steadily, with LR decay steps at epochs 17 and 23 clearly visible as slope changes.

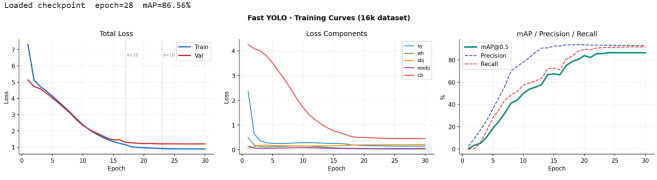


Fig. 4. Fast YOLO training curves on 16,000 scenes. Left: total loss with train/val closely tracking, minimal overfitting gap. Centre: loss components showing cls dominant early then converging. Right: mAP, precision and recall all rising steadily to 86.56%, 92.93%, 91.74%. Best checkpoint at epoch 28 with mAP=86.56%.

D. Pretraining Confusion Matrix

Fig. 5 shows the normalised confusion matrix from backbone pretraining on full MNIST. All digits achieve 90%+ accuracy with Digits 0 and 1 reaching 98%, confirming the backbone learned strong digit features before detection finetuning.

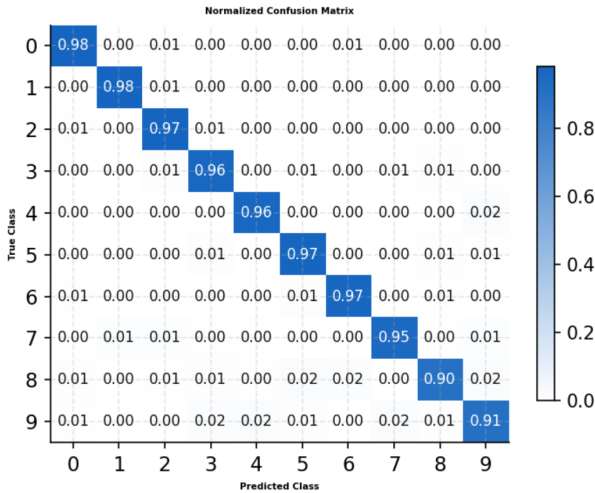


Fig. 5. Normalised confusion matrix from Fast YOLO backbone pretraining on full 60,000-image MNIST. Diagonal values 90–98% confirm strong digit feature learning. Digit 8 shows the most confusion (0.90) consistent with its visual similarity to 0 and 6.

E. Qualitative Detection Results

Fig. 6 presents qualitative Fast YOLO detection results on validation scenes. Green bounding boxes indicate ground-truth annotations, while red dashed boxes represent model predictions with class labels and confidence scores.

The model demonstrates strong localisation performance, with predicted bounding boxes generally aligning well with ground-truth digit positions across most samples. The detector successfully identifies and classifies the majority of digits, with occasional errors occurring in visually ambiguous or overlapping regions.

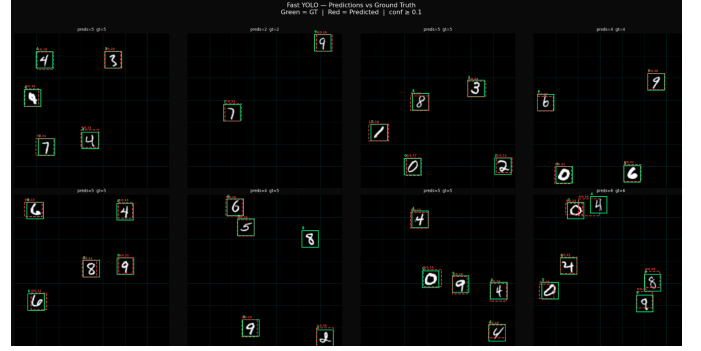


Fig. 6. Fast YOLO detection results on validation scenes. Green boxes denote ground truth annotations, while red dashed boxes represent predicted bounding boxes with class labels and confidence scores. The model shows consistent localisation accuracy and robust digit classification across diverse samples, with minor errors in overlapping or low-contrast regions.

F. Failure Case Analysis

Qualitative inspection of validation predictions reveals three distinct failure categories:

Missing detections. A small fraction of digits are not detected at $\text{IoU} \geq 0.5$. This occurs primarily when a digit centre falls near a grid cell boundary, causing the ground truth assignment to conflict with the cell the model associates with that spatial region. This is a known structural limitation of the one-object-per-cell constraint described in paper Section 2, and is not specific to our implementation.

Classification errors. The most common error type involves correct localisation with incorrect class prediction. Digit pairs most frequently confused are 8 with 0 and 6, and 9 with 4. This is consistent with the pretraining confusion matrix in Fig. 5 where Digits 8 and 9 show the lowest diagonal values (0.90 and 0.91 respectively). The detector inherits the backbone’s residual confusion between visually similar digit shapes.

Low confidence correct detections. Some correctly localised and classified digits produce confidence scores below 0.2. This occurs for unusual handwriting styles including heavily tilted or partially formed strokes that are underrepresented in the 16,000 training scenes. The objectness score reflects the model’s uncertainty about whether a valid object occupies the cell, and unusual appearances push this score toward the detection threshold.

These findings are consistent with the overall precision of 92.93% and recall of 91.74%, indicating that while the vast majority of digits are correctly detected and classified, the remaining errors follow predictable and explainable patterns rooted in dataset characteristics and architectural constraints inherited from the original YOLOv1 design.

VI. DISCUSSION

A. Faithfulness to Paper

The core components of YOLOv1 are faithfully reproduced throughout: the five-term loss function with correct lambda weightings, responsible box selection by IoU, square root transformation for wh losses, the two-stage training pipeline,

SGD with momentum and weight decay, and the step learning rate schedule. Fast YOLO is an explicitly described paper variant, not a deviation. All deviations are documented in Table III.

Inference speed. Fast YOLO achieves 177.26 FPS on our greyscale single-channel input, exceeding the paper’s reported 155 FPS on three-channel VOC images. This is consistent with the reduced input complexity of greyscale MNIST scenes compared to natural colour images, and confirms the paper’s claim that Fast YOLO pushes the boundary of fast object detection.

B. Key Findings

Model capacity vs task complexity. Across our ablation runs, deeper architectures consistently underperformed on the MNIST detection task. A 16-layer variant achieved 19.7% mAP, while 12-layer and 24-layer configurations exhibited overfitting and were stopped early. Fast YOLO (9 conv layers) achieved 38.16% mAP on the same 8,000-scene dataset the best result among all architectural variants. This confirms that MNIST digit detection requires simpler feature hierarchies than the full YOLOv1 backbone was designed for, and that Fast YOLO’s reduced capacity is better matched to the task complexity.

Pretraining data coverage. Using the full 60,000-image MNIST training set rather than a 10,000-sample subset significantly improves detection generalisation by exposing the backbone to all digit variation styles before finetuning.

Weight initialisation. The paper’s normal(0, 0.01) initialisation is incompatible with 24-layer depth, confirming theoretical predictions about vanishing gradients. Kaiming uniform is a necessary modern substitute.

Learning rate. The paper’s LR of 1e-2 produces degenerate predictions on small synthetic data, demonstrating that paper hyperparameters do not transfer directly across dataset scales.

Scale mismatch in pretraining. Matching pretraining digit size (48x48) to detection digit size improved early convergence, reducing val loss at epoch 1 from 7.226 to 6.021.

C. Limitations

The primary limitation is dataset scale relative to the original VOC benchmark. Extending to larger synthetic datasets or real-world detection tasks would provide a more rigorous evaluation of the full 24-layer architecture.

VII. CONCLUSION

We have presented a faithful from-scratch reproduction of YOLOv1 applied to synthetic MNIST digit detection, supported by a systematic nine-run ablation study conducted over 24 hours of GPU experimentation. Our primary result using Fast YOLO the 9-layer paper variant achieves mAP@0.5 of 86.56% at epoch 28 and on 16,000 synthetic scenes with full 60,000-image MNIST backbone pretraining. On identical 8,000-scene datasets, Fast YOLO achieves 38.16% mAP compared to 19.67% for a 16-layer ablation and 4.55% for the full 24-layer architecture, providing empirical evidence that model capacity must match task complexity.

All implementation choices are documented with explicit reference to paper sections, all deviations are justified with empirical evidence from ablation runs, and all experimental results are reproducible from the provided code and checkpoints. The study demonstrates that while the YOLOv1 loss function, training dynamics, and two-stage pipeline transfer faithfully to MNIST detection, architecture selection must account for the fundamental difference in visual complexity between natural image datasets and handwritten digit scenes.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [2] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587.
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1026–1034.